

```

// DealRadar Scraper
// Haalt actuele cashback/gratis deals op van nl.pepper.com
// en slaat ze op in Supabase.

const SUPABASE_URL = process.env.SUPABASE_URL;
const SUPABASE_KEY = process.env.SUPABASE_KEY; // gebruik je publishable key

// ——— Helpers —————

async function fetchPage(url) {
  const res = await fetch(url, {
    headers: {
      "User-Agent": "Mozilla/5.0 (compatible; DealRadarBot/1.0)",
      "Accept-Language": "nl-NL,nl;q=0.9",
      Accept: "text/html",
    },
  });
  if (!res.ok) throw new Error(`HTTP ${res.status} bij ${url}`);
  return res.text();
}

// Eenvoudige HTML parser zonder dependencies
function extractBetween(str, start, end, fromIndex = 0) {
  const s = str.indexOf(start, fromIndex);
  if (s === -1) return null;
  const e = str.indexOf(end, s + start.length);
  if (e === -1) return null;
  return str.slice(s + start.length, e).trim();
}

function stripHtml(html) {
  return html
    .replace(/<[^>]+>/g, " ")
    .replace(/&/g, "&")
    .replace(/&lt;/g, "<")
    .replace(/&gt;/g, ">")
    .replace(/&quot;/g, "'")
    .replace(/&#39;/g, "'")
    .replace(/&nbsp;/g, " ")
    .replace(/\\s+/g, " ")
    .trim();
}

// ——— Detecteer deal-informatie uit tekst —————

```

```

function detectApp(text) {
  const t = text.toLowerCase();
  if (t.includes("woolsocks")) return "Woolsocks";
  if (t.includes("scoupy")) return "Scoupy";
  if (t.includes("shopmium")) return "Shopmium";
  if (t.includes("tikkie")) return "Tikkie";
  return "Overig";
}

function detectStore(text) {
  const stores = [
    "Albert Heijn", "Jumbo", "Lidl", "Aldi", "Plus", "Dirk",
    "Spar", "Coop", "Hoogvliet", "Vomar", "Picnic", "DekaMarkt",
  ];
  for (const store of stores) {
    if (text.toLowerCase().includes(store.toLowerCase())) return store;
  }
  return "Diverse supermarkten";
}

function detectCategory(text) {
  const t = text.toLowerCase();
  if (t.includes("pizza") || t.includes("diepvries") || t.includes("frozen")) return "Frozen";
  if (t.includes("drank") || t.includes("sap") || t.includes("cola") || t.includes("bier")) return "Drank";
  if (t.includes("kaas") || t.includes("zuivel") || t.includes("melk") || t.includes("boter")) return "Kaas";
  if (t.includes("chips") || t.includes("snack") || t.includes("doritos") || t.includes("crispies")) return "Chips";
  if (t.includes("schoonmaak") || t.includes("wasmiddel") || t.includes("drecht")) return "Schoonmaak";
  if (t.includes("shampoo") || t.includes("deodorant") || t.includes("zeep") || t.includes("hygiëne")) return "Hygiëne";
  return "Overig";
}

function detectDealType(text) {
  const t = text.toLowerCase();
  if (t.includes("gratis") || t.includes("100% cashback") || t.includes("helemaal gratis")) return "Gratis";
  if (t.includes("cashback") || t.includes("geld terug") || t.includes("terugbetaald")) return "Cashback";
  if (t.includes("korting") || t.includes("% korting")) return "Korting";
  return "Cashback";
}

function detectCashback(text) {
  // Zoek bedragen zoals €2,49 of €2.49 of 2,49 euro of 50% cashback
  const euroMatch = text.match(/€\s*(\d+[\.,]\d{1,2})/);
  if (euroMatch) return parseFloat(euroMatch[1].replace(",", "."));

  const pctMatch = text.match(/(\d+)\s*\%cashback/i);
  if (pctMatch) return parseFloat(pctMatch[1]) / 100; // sla op als factor, later

```

```

const wordMatch = text.match(/(\d+)[.](\d{2})\s*euro/i);
if (wordMatch) return parseFloat(`${wordMatch[1]}.${wordMatch[2]}`);

return 1.00; // fallback
}

function detectIsFree(text) {
  const t = text.toLowerCase();
  return t.includes("gratis") || t.includes("100% cashback") || t.includes("helem

}

// Einddatum: standaard 14 dagen vanaf nu als we het niet kunnen detecteren
function detectEndDate(text) {
  // Zoek patronen zoals "t/m 30 juni" of "tot 1 juli" of "geldig t/m 30-06"
  const months = { jan: 1, feb: 2, mrt: 3, apr: 4, mei: 5, jun: 6, jul: 7, aug: 8

  const match1 = text.match(/t\/m\s+(\d{1,2})\s+(jan|feb|mrt|apr|mei|jun|jul|aug|
  if (match1) {
    const month = months[match1[2].toLowerCase()];
    const year = new Date().getFullYear();
    return `${year}-${String(month).padStart(2, "0")}-${String(match1[1]).padStar
  }

  const match2 = text.match(/(\d{1,2})-(\d{2})-(\d{4})/);
  if (match2) return `${match2[3]}-${match2[2]}-${match2[1]}`;

  const match3 = text.match(/(\d{4})-(\d{2})-(\d{2})/);
  if (match3) return match3[0];

  // Fallback: 14 dagen vanaf nu
  const d = new Date();
  d.setDate(d.getDate() + 14);
  return d.toISOString().split("T")[0];
}

// ——— Pepper scraper ———

async function scrapePepper() {
  console.log("🔍 Pepper.com boodschappen deals ophalen...");
  const html = await fetchPage("https://nl.pepper.com/groep/boodschappen");
  const deals = [];

  // Pepper gebruikt article tags met class "thread"
  // We splitten op deal-blokken
  const articlePattern = /<article[^\>]*class="[^"]*thread"[^\>]*>([\s\S]*)<\
  let match;

```

```

while ((match = articlePattern.exec(html)) !== null) {
  try {
    const block = match[1];

    // Titel
    const titleRaw = extractBetween(block, 'class="thread-title">', "</");
    if (!titleRaw) continue;
    const title = stripHtml(titleRaw);
    if (!title || title.length < 5) continue;

    // Link
    const linkMatch = block.match(/href="(https:\/\/nl\.pepper\.com\/aanbieding
    const link = linkMatch ? linkMatch[1] : "https://nl.pepper.com/groep/boodsc

    // Beschrijving (voor detectie)
    const descRaw = extractBetween(block, 'class="thread-description">', "</div
    const desc = descRaw ? stripHtml(descRaw) : "";
    const fullText = `${title} ${desc}`;

    // Sla deals over die niet over cashback/gratis gaan
    const relevant = ["cashback", "gratis", "geld terug", "scoupy", "woolsocks"
      .some(kw => fullText.toLowerCase().includes(kw));
    if (!relevant) continue;

    const app = detectApp(fullText);
    const store = detectStore(fullText);
    const category = detectCategory(fullText);
    const deal_type = detectDealType(fullText);
    const cashback = detectCashback(fullText);
    const is_free = detectIsFree(fullText);
    const end_date = detectEndDate(fullText);

    deals.push({
      name: title.slice(0, 200),
      store,
      app,
      category,
      deal_type,
      discount: cashback,
      cashback,
      end_date,
      conditions: desc.slice(0, 500) || "Zie de originele deal voor de voorwaar
      action_url: link,
      is_free,
      source: "pepper",
    });
  }
}

```

```

    } catch (e) {
      // Skip kapotte blokken
    }
  }

  console.log(`✅ ${deals.length} deals gevonden op Pepper`);
  return deals;
}

// — Gratis247 scraper —————

async function scrapeGratis247() {
  console.log("🔍 Gratis247.nl deals ophalen...");
  const html = await fetchPage("https://gratis247.nl/category/cashback/");
  const deals = [];

  const articlePattern = /<article[^>]*>([\s\S]*?)<\/article>/gi;
  let match;

  while ((match = articlePattern.exec(html)) !== null) {
    try {
      const block = match[1];

      const titleRaw = extractBetween(block, "<h2", "</h2>");
      if (!titleRaw) continue;
      const title = stripHtml(titleRaw);
      if (!title || title.length < 5) continue;

      const linkMatch = block.match(/href="(https:\\\\gratis247\\.nl\\/[^"]+)"/);
      const link = linkMatch ? linkMatch[1] : "https://gratis247.nl";

      const descRaw = extractBetween(block, "<p", "</p>");
      const desc = descRaw ? stripHtml(descRaw) : "";
      const fullText = `${title} ${desc}`;

      const relevant = ["cashback", "gratis", "geld terug", "scoupy", "woolsocks"]
        .some(kw => fullText.toLowerCase().includes(kw));
      if (!relevant) continue;

      deals.push({
        name: title.slice(0, 200),
        store: detectStore(fullText),
        app: detectApp(fullText),
        category: detectCategory(fullText),
        deal_type: detectDealType(fullText),
        discount: detectCashback(fullText),
        cashback: detectCashback(fullText),
      });
    } catch (e) {
      console.log("Error parsing article block:", e);
    }
  }
}

```

```

        end_date: detectEndDate(fullText),
        conditions: desc.slice(0, 500) || "Zie de originele deal voor de voorwaar
        action_url: link,
        is_free: detectIsFree(fullText),
        source: "gratis247",
    });
} catch (e) {}
}

console.log(`✅ ${deals.length} deals gevonden op Gratis247`);
return deals;
}

// — Supabase opslaan —————

async function upsertDeals(deals) {
    if (!SUPABASE_URL || !SUPABASE_KEY) {
        console.log("⚠️ Geen Supabase config. Deals worden geprint:\n");
        deals.forEach(d => console.log(` • ${d.name} (${d.app}) - €${d.cashback} - $
        return;
    }

    console.log(`📁 ${deals.length} deals opslaan in Supabase...`);

    // Verwijder verlopen deals eerst
    const today = new Date().toISOString().split("T")[0];
    await fetch(`${SUPABASE_URL}/rest/v1/deals?end_date=lt.${today}&source=eq.peppe
        method: "DELETE",
        headers: {
            apikey: SUPABASE_KEY,
            Authorization: `Bearer ${SUPABASE_KEY}`,
        },
    });

    await fetch(`${SUPABASE_URL}/rest/v1/deals?end_date=lt.${today}&source=eq.grati
        method: "DELETE",
        headers: {
            apikey: SUPABASE_KEY,
            Authorization: `Bearer ${SUPABASE_KEY}`,
        },
    });

    // Deals in batches van 20 opslaan
    const batchSize = 20;
    let saved = 0;

    for (let i = 0; i < deals.length; i += batchSize) {
        const batch = deals.slice(i, i + batchSize);
    }

```

```

const res = await fetch(`${SUPABASE_URL}/rest/v1/deals`, {
  method: "POST",
  headers: {
    apikey: SUPABASE_KEY,
    Authorization: `Bearer ${SUPABASE_KEY}`,
    "Content-Type": "application/json",
    Prefer: "resolution=ignore-duplicates,return=minimal",
  },
  body: JSON.stringify(batch),
});

if (res.ok || res.status === 201 || res.status === 200) {
  saved += batch.length;
} else {
  const err = await res.text();
  console.error(`❌ Batch fout: ${err}`);
}
}

console.log(`✅ ${saved} deals opgeslagen`);
}

// — Hoofdfunctie —————

async function main() {
  console.log("🚀 DealRadar Scraper gestart:", new Date().toLocaleString("nl-NL"));

  const results = await Promise.allSettled([scrapePepper(), scrapeGratis247()]);

  const allDeals = [];
  for (const result of results) {
    if (result.status === "fulfilled") {
      allDeals.push(...result.value);
    } else {
      console.error("❌ Scraper fout:", result.reason.message);
    }
  }

  // Deduplicateer op naam (lowercase)
  const seen = new Set();
  const unique = allDeals.filter(d => {
    const key = d.name.toLowerCase().slice(0, 50);
    if (seen.has(key)) return false;
    seen.add(key);
    return true;
  });
}

```

```
    console.log(`📦 Totaal unieke deals: ${unique.length}`);  
    await upsertDeals(unique);  
  
    console.log("✅ Klaar!\n");  
}  
  
main().catch(err => {  
    console.error("💥 Fatale fout:", err);  
    process.exit(1);  
});
```